Plan działania — poprawki projektu web-dev-app

> Podstawa: analiza kodu źródłowego z 27.03.2026 (`docs/project-analysis.md`) Zakres: błędy krytyczne, refaktoryzacja, jakość kodu, UX, rozbudowa testów

Spis treści

1. Priorytety na pierwszy rzut
2. Priorytet 1 — Błędy krytyczne (naprawić natychmiast)
3. Priorytet 2 — Refaktoryzacja i DX
4. Priorytet 3 — UX i dopracowanie produktu
5. Priorytet 4 — Testy i jakość
6. Priorytet 5 — Nowe funkcjonalności
7. Mapa zależności między zadaniami

Priorytety na pierwszy rzut

#	Zadanie	Priorytet	Trudność	Plik(i)
P1.1	Napraw migrację `MODIFY COLUMN` (blokuje wszystkie testy)	● Krytyczny	Łatwe	`database/migrations/2026_03_22_194527_update_project_phases_status_enum.php`
P1.2	Guard dla `Setting::get()` w middleware (HTTP 500 w ExampleTest)	● Krytyczny	Łatwe	`app/Http/Middleware/HandleInertiaRequests.php`
P1.3	Napraw fałszywy sukces w kalkulatorze wyceny	● Krytyczny	Łatwe	`resources/js/Components/Marketing/CostCalculatorV2.jsx`
P1.4	Napraw pole `tax_amount` → `vat_amount` w eksporcie faktur	● Krytyczny	Łatwe	`app/Http/Controllers/ReportController.php`
P1.5	Napraw filtrowanie leadów po `status` zamiast `pipeline_stage_id`	● Krytyczny	Łatwe	`app/Http/Controllers/ReportController.php`
P2.1	Rozbij `PortalController` na moduły	● Wysoki	Średnie	`app/Http/Controllers/PortalController.php`
P2.2	Rozbij `ProcessAutomationJob` na warstwy	● Wysoki	Średnie	`app/Jobs/ProcessAutomationJob.php`
P2.3	Wydziel inline JS z `AdminPanelProvider`	● Wysoki	Łatwe	`app/Providers/Filament/AdminPanelProvider.php`
P2.4	Dodaj skrypty jakościowe do `package.json`	● Wysoki	Łatwe	`package.json`
P3.1	Zastąp placeholder w `Dashboard.jsx` właściwym dashboardem	○ Średni	Średnie	`resources/js/Pages/Dashboard.jsx`
P3.2	Ujednolicenie copy i lokalizacji	○ Średni	Żmudne	`resources/js/Pages/`, `resources/js/Components/`
P3.3	Obsługa pustych stanów i błędów w portalu	○ Średni	Łatwe	`resources/js/Pages/Portal/`

#	Zadanie	Priorytet	Trudność	Plik(i)
P4.1	Napraw środowisko testowe i uzupełnij testy portalu	⚪ Średni	Średnie	`tests/Feature/`
P5.1	Timeline aktywności klienta	⚫ Niski	Duże	nowe pliki
P5.2	Dashboard operacyjny zespołu	⚫ Niski	Duże	nowe pliki

Priorytet 1 — Błędy krytyczne

P1.1 — Migracja `MODIFY COLUMN` niezgodna z SQLite

Problem: Plik `database/migrations/2026_03_22_194527_update_project_phases_status_enum.php` używa surowego SQL `ALTER TABLE ... MODIFY COLUMN`, który jest składnią MySQL i nie działa na SQLite in-memory używanym w testach. Blokuje to uruchomienie całego zestawu testów.

Jak naprawić:

Zamienić surowe `DB::statement()` na warunkowe wykonanie z detekcją drivera bazy, albo — lepiej — przepisać migrację na podejście kompatybilne z obydwoma silnikami.

```
// database/migrations/2026_03_22_194527_update_project_phases_status_enum.php

public function up(): void
{
    $driver = DB::getDriverName();

    if ($driver === 'mysql' || $driver === 'mariadb') {
        DB::statement("ALTER TABLE project_phases MODIFY COLUMN status
ENUM('pending','active','in_progress','completed','cancelled') NOT NULL DEFAULT 'pending'");
        DB::table('project_phases')->where('status', 'active')->update(['status' => 'in_progress']);
        DB::statement("ALTER TABLE project_phases MODIFY COLUMN status
ENUM('pending','in_progress','completed','cancelled') NOT NULL DEFAULT 'pending'");
    } else {
        // SQLite nie obsługuje MODIFY COLUMN — dane migrujemy przez UPDATE, typ kolumny zostawiamy string
        DB::table('project_phases')->where('status', 'active')->update(['status' => 'in_progress']);
    }
}

public function down(): void
{
    $driver = DB::getDriverName();

    if ($driver === 'mysql' || $driver === 'mariadb') {
        DB::statement("ALTER TABLE project_phases MODIFY COLUMN status
ENUM('pending','active','in_progress','completed','cancelled') NOT NULL DEFAULT 'pending'");
        DB::table('project_phases')->where('status', 'in_progress')->update(['status' => 'active']);
        DB::statement("ALTER TABLE project_phases MODIFY COLUMN status ENUM('pending','active','completed') NOT NULL
DEFAULT 'pending'");
    } else {
        DB::table('project_phases')->where('status', 'in_progress')->update(['status' => 'active']);
    }
}
```

Weryfikacja: `php artisan test` nie powinien już failować na etapie migracji.

P1.2 — `Setting::get()` w middleware powoduje HTTP 500 w testach

Problem: `HandleInertiaRequests::share()` bezwarunkowo wywołuje `Setting::get()` dla każdego żądania. Przy testach z refreshDatabase lub bazowym `ExampleTest` tabela `settings` może nie istnieć, co powoduje wyjątek i odpowiedź 500.

Jak naprawić:

Owinąć blok `tracking` w try/catch lub dodać sprawdzenie czy tabela istnieje przed odczytem.

```php
// app/Http/Middleware/HandleInertiaRequests.php

'tracking' => $this->resolveTrackingSettings(),

// dodać metodę pomocniczą:
private function resolveTrackingSettings(): array
{
    try {
        return [
            'gtm_enabled'            => (bool) Setting::get('gtm_enabled', false),
            'gtm_id'                 => Setting::get('gtm_id', ''),
            'ga4_enabled'            => (bool) Setting::get('ga4_enabled', false),
            'ga4_id'                 => Setting::get('ga4_id', ''),
            'pixel_enabled'          => (bool) Setting::get('pixel_enabled', false),
            'pixel_id'               => Setting::get('pixel_id', ''),
            'gads_enabled'           => (bool) Setting::get('gads_enabled', false),
            'gads_id'                => Setting::get('gads_id', ''),
            'cookie_consent_enabled' => (bool) Setting::get('cookie_consent_enabled', true),
        ];
    } catch (\Throwable) {
        return [
            'gtm_enabled' => false, 'gtm_id' => '',
            'ga4_enabled' => false, 'ga4_id' => '',
            'pixel_enabled' => false, 'pixel_id' => '',
            'gads_enabled' => false, 'gads_id' => '',
            'cookie_consent_enabled' => true,
        ];
    }
}
```

Weryfikacja: `ExampleTest` powinien zwracać 200 zamiast 500.

P1.3 — Fałszywy sukces w kalkulatorze wyceny

Problem: W `CostCalculatorV2.jsx` metoda `handleSubmit` wywołuje `setSubmitted(true)` w bloku `finally`, czyli sukces jest pokazywany nawet jeśli `fetch()` rzucił wyjątek. Użytkownik dostaje fałszywe potwierdzenie wysłania zapytania.

Plik: `resources/js/Components/Marketing/CostCalculatorV2.jsx`, linie ~156–183

Jak naprawić:

Przenieść `setSubmitted(true)` do bloku `try` (po `await fetch`), a do `catch` dodać ustawienie stanu błędu widocznego dla użytkownika.

```jsx
const handleSubmit = useCallback(async () => {
    if (!a.contactEmail || submitting) return;
    setSubmitting(true);
    try {
        const csrf = document.querySelector('meta[name="csrf-token"]')?.content ?? '';
        const res = await fetch(route('calculator.lead'), {
            method: 'POST',
            headers: { 'Content-Type': 'application/json', 'X-CSRF-TOKEN': csrf },
            body: JSON.stringify({
                ...a,
                estimateLow:  estimate?.low,
                estimateHigh: estimate?.high,
            }),
        });
        if (!res.ok) throw new Error('server_error');
        pushEvent('generate_lead', {
            lead_source:  'calculator_v2',
            project_type: a.projectType,
            estimate_low:  estimate?.low,
            estimate_high: estimate?.high,
        });
        if (typeof window.fbq === 'function') window.fbq('track', 'Lead');
        setSubmitted(true);
    } catch (_) {
        setSubmitError(true); // nowy stan — dodać: const [submitError, setSubmitError] = useState(false);
    } finally {
        setSubmitting(false);
```

```
    }
}, [a, estimate, submitting]);
```

W JSX dodać komunikat błędu przy `submitError === true` (np. "Wystąpił problem. Spróbuj ponownie lub skontaktuj się z nami.").

Weryfikacja: Przy symulowanym błędzie sieciowym/serwerowym nie może pojawić się ekran sukcesu.

P1.4 — Pole `tax_amount` zamiast `vat_amount` w eksporcie faktur

Problem: W `ReportController::invoicesSpreadsheet()` linia:

```
$sheet->setCellValueByColumnAndRow(8, $row + 2, $invoice->tax_amount ?? 0);
```

Model `Invoice` używa pola `vat_amount`, nie `tax_amount`. Eksport XLSX/CSV faktur zwraca 0 w kolumnie VAT.

Jak naprawić:

```
// app/Http/Controllers/ReportController.php — metoda invoicesSpreadsheet()
$sheet->setCellValueByColumnAndRow(8, $row + 2, $invoice->vat_amount ?? 0);
```

Weryfikacja: Pobrać raport faktur jako XLSX, sprawdzić czy kolumna VAT zawiera rzeczywiste wartości.

P1.5 — Filtrowanie leadów po nieistniejącym polu `status`

Problem: W `ReportController::leads()` zapytanie używa:

```
->when($request->status, fn ($q) => $q->where('status', $request->status))
```

Leady w systemie nie mają prostego pola `status` dla etapu lejka — posługują się relacją `pipeline_stage_id` i modelem `PipelineStage`. Filtrowanie po `status` nie działa zgodnie z intencją lub zwraca błędne wyniki.

Jak naprawić:

```
// app/Http/Controllers/ReportController.php — metoda leads()
->when($request->stage_id, fn ($q) => $q->where('pipeline_stage_id', $request->stage_id))
```

Dodatkowo zaktualizować nagłówek kolumny `Status` → `Stage` w `leadsSpreadsheet()`:

```
$headers = ['ID', 'Name', 'Email', 'Phone', 'Company', 'Source', 'Stage', 'Stage Name', 'Value', 'Created'];
// ...
$sheet->setCellValueByColumnAndRow(7, $row + 2, $lead->pipeline_stage_id ?? '');
$sheet->setCellValueByColumnAndRow(8, $row + 2, $lead->stage?->name ?? '');
```

Weryfikacja: Filtr na stronie raportów leadów powinien operować na stage_id, a eksport powinien zwracać nazwę etapu.

Priorytet 2 — Refaktoryzacja i DX

P2.1 — Rozbicie `PortalController` na mniejsze kontrolery

Problem: `PortalController` obsługuje dashboard, projekty, wiadomości, faktury, quotes, kontrakty, płatności i ustawienia powiadomień — około 300+ linii, wiele odpowiedzialności w jednej klasie. Trudno testować i rozszerzać.

Plan:

Podzielić na osobne kontrolery w namespace `App\Http\Controllers\Portal\`:

```
app/Http/Controllers/Portal/
  DashboardController.php    — metoda index(), dane podsumowania
  ProjectController.php      — show(), messages(), sendMessage()
```

```
  InvoiceController.php      — index(), show(), download()
  QuoteController.php        — index(), show(), accept(), reject()
  ContractController.php     — show(), sign(), accept()
  PaymentController.php      — stripeIntent(), stripeConfirm(), payuInit(), payuReturn()
  NotificationController.php — settings(), updateSettings()
```

Obecny `PortalController` może na razie pozostać jako fasada delegująca, albo trasy w `routes/web.php` mogą być zaktualizowane bezpośrednio.

Kolejność:

1. Wynieść `PaymentController` (najbardziej izolowany)
2. Wynieść `NotificationController`
3. Wynieść `ContractController`
4. Wynieść `QuoteController`
5. Wynieść `InvoiceController`
6. Wynieść `ProjectController`
7. Pozostawić `DashboardController` jako uproszczony

Zmiana w routach (`routes/web.php`):

```php
// Przed:
Route::controller(PortalController::class)->group(function () { ... });

// Po:
Route::prefix('portal')->name('portal.')->middleware(['auth', 'portal'])->group(function () {
    Route::get('/', [Portal\DashboardController::class, 'index'])->name('dashboard');
    Route::get('/projects/{project}', [Portal\ProjectController::class, 'show'])->name('projects.show');
    // ...
});
```

P2.2 — Rozbicie `ProcessAutomationJob` na warstwy

Problem: `ProcessAutomationJob` robi trzy rzeczy naraz: wybiera reguły, ocenia warunki i wykonuje akcje (8 typów). Klasa ~300 linii. Trudna do testowania jednostkowego i rozszerzania o nowe typy akcji.

Plan:

Wydzielić dedykowane klasy akcji:

```
app/Automation/
  Actions/
    SendEmailAction.php
    SendInternalEmailAction.php
    SendSmsAction.php
    NotifyAdminAction.php
    AddTagAction.php
    ChangeStatusAction.php
    CreatePortalAccessAction.php
  ConditionEvaluator.php
  AutomationActionContract.php  (interface)
```

`ProcessAutomationJob::executeAction()` staje się:

```php
private function executeAction(array $action): void
{
    $map = [
        'send_email'           => Actions\SendEmailAction::class,
        'send_internal_email'  => Actions\SendInternalEmailAction::class,
        'send_sms'             => Actions\SendSmsAction::class,
        'notify_admin'         => Actions\NotifyAdminAction::class,
        'add_tag'              => Actions\AddTagAction::class,
        'change_status'        => Actions\ChangeStatusAction::class,
        'create_portal_access' => Actions\CreatePortalAccessAction::class,
    ];

    $class = $map[$action['type'] ?? ''] ?? null;
```

```
    if (!$class) return;

    app($class)->execute($action, $this->context);
}
```

`ConditionEvaluator` staje się osobną, testowalną klasą:

```
class ConditionEvaluator
{
    public function evaluate(array $conditions, array $context): bool
    {
        foreach ($conditions as $condition) { ... }
        return true;
    }
}
```

P2.3 — Wydzielenie inline JS z `AdminPanelProvider`

Problem: `AdminPanelProvider` zawiera ~100-liniowy blok JavaScript z obsługą dźwięków powiadomień, czytaniem i usuwaniem powiadomień z bazy. Kod JS wstrzykiwany jest przez PHP przez `renderHook(BODY_END, ...)`. Utrudnia to debugowanie JS i testowanie.

Plan:

1. Wyekstrahować kod JS do `resources/js/admin/notifications.js`
2. Zarejestrować go przez Vite jako osobny entry point lub dołączyć do `app.js`
3. W `AdminPanelProvider` zastąpić inline JS tylko wstrzyknięciem danych (config-only):

```
// AdminPanelProvider.php
->renderHook(PanelsRenderHook::BODY_END, fn () => new HtmlString(
    '<script>window.AdminConfig = ' . json_encode([
        'notificationSoundUrl' => asset('sounds/gg-ping.mp3'),
        'notificationReadUrl'  => route('api.notifications.read'),
    ]) . '</script>'
))
```

P2.4 — Skrypty jakościowe w `package.json`

Problem: `package.json` zawiera tylko skrypty `build` i `dev`. Brak `lint`, `format`, `typecheck`. Nie ma żadnej automatycznej kontroli jakości kodu frontendowego.

Plan:

```
{
  "scripts": {
    "dev": "vite",
    "build": "vite build",
    "lint": "eslint resources/js --ext .js,.jsx --max-warnings 0",
    "lint:fix": "eslint resources/js --ext .js,.jsx --fix",
    "format": "prettier --write resources/js",
    "format:check": "prettier --check resources/js"
  },
  "devDependencies": {
    "eslint": "^9.x",
    "eslint-plugin-react": "^7.x",
    "eslint-plugin-react-hooks": "^5.x",
    "prettier": "^3.x"
  }
}
```

Dodać `.eslintrc.json`:

```
{
  "extends": ["eslint:recommended", "plugin:react/recommended", "plugin:react-hooks/recommended"],
```

```
  "env": { "browser": true, "es2022": true },
  "settings": { "react": { "version": "detect" } },
  "rules": {
    "no-unused-vars": "warn",
    "react/prop-types": "off"
  }
}
```

Dodać `.prettierrc`:

```
{
  "semi": true,
  "singleQuote": true,
  "tabWidth": 4,
  "trailingComma": "es5",
  "printWidth": 120
}
```

P2.5 — Ujednolicenie tworzenia leadów (formularz vs kalkulator)

Problem: Lead powstaje dwoma ścieżkami: `ContactController` i `CalculatorLeadController`. Logika tworzenia leada i ewentualne notyfikacje są zduplikowane.

Plan:

1. Wynieść wspólną logikę do `app/Actions/CreateLeadAction.php`:

```
class CreateLeadAction
{
    public function execute(array $data, string $source): Lead
    {
        $lead = Lead::create([
            'name'   => $data['name'],
            'email'  => $data['email'],
            'source' => $source,
            // ...
        ]);

        event(new LeadCreated($lead));
        return $lead;
    }
}
```

2. Oba kontrolery używają `CreateLeadAction`.

Priorytet 3 — UX i dopracowanie produktu

P3.1 — Zastąpienie placeholder `Dashboard.jsx`

Problem: `resources/js/Pages/Dashboard.jsx` to dosłownie domyślny widok Breeze z nagłówkiem "You're logged in!" — brak jakichkolwiek informacji biznesowych dla zalogowanego użytkownika niebędącego klientem portalu.

Plan:

Zaprojektować i zaimplementować właściwy dashboard dla pracownika agencji zawierający:

- liczba aktywnych projektów
- liczba otwartych leadów na dzień
- liczba niezapłaconych faktur (overdue + sent)
- szybki podgląd ostatnich aktywności (ostatnie leady, projekty z deadline'em w ciągu 7 dni)
- skróty do najczęściej używanych sekcji Filament (opcjonalne, przez linki)

```
// resources/js/Pages/Dashboard.jsx
// Props przekazywane z DashboardController (backend do zbudowania):
// { activeProjects, openLeads, unpaidInvoices, upcomingDeadlines }
```

```
export default function Dashboard({ activeProjects, openLeads, unpaidInvoices, upcomingDeadlines }) {
    return (
        <AuthenticatedLayout>
            <div className="grid grid-cols-3 gap-6">
                <StatCard label="Aktywne projekty" value={activeProjects} />
                <StatCard label="Otwarte leady" value={openLeads} />
                <StatCard label="Niezapłacone faktury" value={unpaidInvoices} />
            </div>
            <UpcomingDeadlinesTable items={upcomingDeadlines} />
        </AuthenticatedLayout>
    );
}
```

Wymagane: zaktualizowanie `DashboardController@index` (lub osobny kontroler) w Laravel, który przekaże te dane przez Inertia.

P3.2 — Ujednolicenie języka i copy

Problem: Spora część copy w komponentach marketingowych i portalu jest twardo wpisana po angielsku, mimo istnienia systemu locale i mechanizmów tłumaczeń przekazywanych przez backend.

Plan działania:

1. Audyt komponentów — znaleźć wszystkie twardo wpisane stringi:

   ```
   grep -r "className" resources/js/Pages/Portal --include="*.jsx" | grep -i '"[A-Z]'
   ```

2. Skategoryzować stringi według lokalizacji: portal klienta, strona marketingowa, Filament (Blade)
3. Strona marketingowa:
   - Stringi już zdefiniowane jako `CalculatorString` w bazie — upewnić się, że wszystkie teksty są przez `s()` helper a nie hardcode
   - Brakujące: dodać do `CalculatorString` rekordy dla komunikatów błędów, CTA itp.
4. Portal klienta:
   - Dodać plik `/lang/pl/portal.php` (i en, pt) z tłumaczeniami tekstów portalu
   - W React korzystać z istniejącego mechanizmu przekazywania locale przez `HandleInertiaRequests`
   - Każdy komponent portalu powinien otrzymywać `translations` prop lub korzystać z hooka `useTrans()`
5. Priorytetowe ekrany do przetłumaczenia: `Portal/Dashboard.jsx`, `Portal/Project.jsx`, `Portal/Contract.jsx`, status messages, nagłówki tabel

---

P3.3 — Obsługa pustych stanów i błędów w portalu

Problem: Część ekranów portalu nie ma przemyślanej obsługi pustego stanu (np. brak projektów, brak faktur) ani stanów błędów (np. failed payment).

Plan:

Dla każdego ekranu portalu dodać:

Portal/Dashboard.jsx:

```
{projects.length === 0 && (
    <EmptyState
        icon={<FolderIcon />}
        title="Brak aktywnych projektów"
        description="Twoje projekty pojawią się tutaj po uruchomieniu realizacji."
    />
)}
```

Portal/Project.jsx:

- komunikat gdy brak wiadomości w wątku
- komunikat gdy brak faz / zadań

Portal/PayInvoice.jsx:

- komunikat błędu po nieudanej płatności (aktualnie brak widoku błędu po powrocie z PayU z `status=FAILED`)

- loading spinner dla obydwu metod już istnieje — sprawdzić czy error path jest obsłużony

Portal/Contract.jsx:

- po podpisaniu: wyraźne potwierdzenie z numerem kontraktu i możliwością pobrania PDF
- przy niepowodzeniu podpisu: komunikat i możliwość powtórzenia

Stworzyć komponent `EmptyState` wielokrotnego użytku w `resources/js/Components/Shared/EmptyState.jsx`.

P3.4 — Obsługa powrotu z płatności PayU (sukces / błąd)

Problem: `PayuService` implementuje inicjację płatności i `returnUrl`, ale brak dedykowanego widoku potwierdzenia lub błędu płatności po powrocie z bramki PayU.

Plan:

1. Zweryfikować trasę `portal.payu.return` w `routes/web.php`
2. W `PortalController` (lub przyszłym `Portal\PaymentController`) upewnić się, że metoda obsługi powrotu:
   - weryfikuje status zamówienia w PayU API
   - przy `COMPLETED` → aktualizuje `Invoice::status` na `paid`
   - przy `FAILED` → przekierowuje na ekran błędu z parametrem błędu
3. Dodać stronę React `Portal/PaymentResult.jsx` z dwiema gałęziami (success / error)

Priorytet 4 — Testy i jakość

P4.1 — Napraw środowisko testowe i przywróć test suite

Kroki:

1. Wykonać poprawkę P1.1 (migracja SQLite) i P1.2 (guard Setting)
2. Uruchomić `php artisan test` — sprawdzić jaki baseline przechodzi
3. Naprawić wszystkie pozostałe flakujące testy
4. Upewnić się, że `FullLeadWorkflowTest` przechodzi od T1 do T8

P4.2 — Testy dla portalu klienta

Problem: Brak testów feature dla portalowych akcji klienta (podpis kontraktu, płatność, pobieranie faktur PDF, akceptacja quote).

Plan — stworzyć pliki:

```
tests/Feature/Portal/
  PortalDashboardTest.php     — dostęp z/bez portal_user_id
  PortalContractSignTest.php  — signing workflow (draw + checkbox), walidacja IP
  PortalInvoicePayTest.php    — inicjacja Stripe intent, zwrot po PayU
  PortalQuoteAcceptTest.php   — accept/reject quote przez portal
  PortalMessagesTest.php      — wysyłanie wiadomości projektowych
```

Szablon struktury testu portalu:

```php
class PortalContractSignTest extends TestCase
{
    use RefreshDatabase;

    public function test_client_can_sign_contract(): void
    {
        $user   = User::factory()->create();
        $client = Client::factory()->create(['portal_user_id' => $user->id]);
        $contract = Contract::factory()->create(['client_id' => $client->id, 'status' => 'sent']);

        $this->actingAs($user)
             ->post(route('portal.contracts.sign', $contract), [
                 'signature_data' => '...',
                 'accept_terms'   => true,
             ])
             ->assertRedirect();

        $this->assertDatabaseHas('contracts', [
```

```
            'id'     => $contract->id,
            'status' => 'signed',
        ]);
    }
}
```

P4.3 — Testy dla raportów

Plan:

```
tests/Feature/
  ReportLeadsTest.php     — html/pdf/xlsx/csv format, filtr stage_id
  ReportInvoicesTest.php  — walidacja pola vat_amount w exporcie
  ReportProjectsTest.php  — html/pdf/xlsx/csv format
```

P4.4 — Testy dla automatyzacji

Plan:

```
tests/Unit/
  ConditionEvaluatorTest.php  — warunki: =, !=, >, <, contains
tests/Feature/
  AutomationTriggerTest.php   — event lead.created → dispatches job
  AutomationActionTest.php    — send_email action wykonuje Mail::queue
```

P4.5 — Testy dla kalkulatora wyceny

Plan:

```
tests/Feature/
  CalculatorLeadTest.php   — POST /calculator/lead walidacja, zapis do DB
tests/Unit/
  CostCalculatorV2Test.jsx — handleSubmit pokazuje błąd przy failed fetch
                           — setSubmitted(true) NIE jest wywoływane przy błędzie
```

Dla frontu — dodać Vitest lub Jest:

```
// package.json scripts
"test": "vitest",
"test:ui": "vitest --ui"
```

Priorytet 5 — Nowe funkcjonalności

P5.1 — Timeline aktywności klienta

Wartość: Klient widzi pełną historię: kiedy lead został stworzony, kiedy wysłano quote, kiedy projekt wystartował, kiedy faktura została opłacona.

Plan:

Backend:

- Tabela `client_activity_log` (id, client_id, event_type, description, created_at)
- Listener zapisujący do tabeli przy kluczowych zdarzeniach (lead.created, quote.sent, contract.signed, invoice.paid, project.started)
- Endpoint `GET /portal/timeline` zwracający chronologiczną listę

Frontend:

- Komponent `TimelineItem` w `resources/js/Components/Portal/`
- Osobna sekcja na `Portal/Dashboard.jsx` lub dedykowana strona `/portal/activity`

P5.2 — Dashboard operacyjny zespołu

Wartość: Zespół agencji widzi w jednym miejscu: projekty z deadline'em, opóźnione faktury, leady nieobsłużone przez X dni, kolejka automatyzacji.

Plan:

Backend:

- Nowy widget Filament `OperationalOverviewWidget`
- Zapytania agregujące: projekty overdue (deadline < now), faktury overdue, leady starsze niż 7 dni bez aktywności

Frontend (Filament):

- Widget tabeli na stronie głównej panelu z sortowaniem i filtrowaniem
- Alerty kolorami: czerwony = overdue, pomarańczowy = approaching, zielony = ok

---

P5.3 — Automatyczne przypomnienia o leadach i zaległościach

Wartość: System sam wysyła przypomnienie do agencji gdy lead nie był obsłużony przez N dni. Wysyła klientowi reminder przed due date faktury.

Plan:

Backend:

- Nowa reguła automatyzacji: `lead.inactive_days` (nowy trigger w `AutomationEventListener`)
- Scheduled command `CheckStaleLeads` uruchamiany codziennie przez `schedule:run`
- Osobna reguła `invoice.due_soon` (np. 3 dni przed due_date) — nieblokująca dla klienta

Filament:

- Nowy trigger dostępny w `AutomationRuleResource` — `lead.inactive_days` z konfiguracją `threshold_days`
- Nowy trigger `invoice.due_soon` z konfiguracją `days_before_due`

P5.4 — Raporty konwersji i źródeł leadów

Wartość: Widok który źródło (organic, cal, referral, paid) generuje leady i jaki procent przechodzi przez cały lejek.

Plan:

Backend:

- Nowy endpoint `GET /admin/reports/conversion`
- Agregacja: `Lead::groupBy('source')->selectRaw('source, count(*) as total')` z join do projects i invoices
- Eksport CSV dostępny przez ten sam endpoint z `?format=csv`

Filament:

- Nowy zasób lub strona `ConversionReportPage` w grupie Reports
- Tabela z kolumnami: Source, Leads, Converted to Project, Conversion Rate, Total Revenue

---

Mapa zależności między zadaniami

```
P1.1 (migracja SQLite)
  ↳ P4.1 (stabilny test suite)
      ↳ P4.2 (testy portalu)
      ↳ P4.3 (testy raportów)
      ↳ P4.4 (testy automatyzacji)

P1.2 (guard Setting)
  ↳ P4.1 (ExampleTest przechodzi)

P1.3 (kalkulator) — samodzielne
P1.4 (vat_amount) — samodzielne
P1.5 (filtr leadów) — samodzielne

P2.1 (rozbij PortalController)
  ↳ P4.2 (łatwiejsze do testowania po podziale)
  ↳ P3.3 (puste stany — łatwiejsze per kontroler)
  ↳ P3.4 (PayU return — własny kontroler)
```

```
P2.2 (rozbij ProcessAutomationJob)
  ↳ P4.4 (testy automatyzacji per action)
  ↳ P5.3 (nowe typy akcji łatwiejsze do dodania)

P2.4 (skrypty jakościowe)
  ↳ P4.5 (frontend testy)

P3.1 (dashboard placeholder)
  ↳ P5.2 (rozbudowa dashboardu)

P5.1 (timeline)
  ↳ P5.4 (raporty konwersji korzystają z tych samych zdarzeń)
```

Estymacja nakładu pracy

Priorytet	Zadania	Szacowany nakład
Priorytet 1 (błędy krytyczne)	P1.1–P1.5	2–4 godziny
Priorytet 2 (refaktoryzacja + DX)	P2.1–P2.5	1–2 dni
Priorytet 3 (UX)	P3.1–P3.4	1–2 dni
Priorytet 4 (testy)	P4.1–P4.5	2–3 dni
Priorytet 5 (nowe funkcje)	P5.1–P5.4	4–6 dni
Łącznie		**~10–13 dni roboczych**

Dokument wygenerowany automatycznie na podstawie analizy kodu źródłowego. Zaktualizować po wdrożeniu każdego zadania.